

Python - Sets

Sets in Python

In Python, a set is an unordered collection of unique elements. Unlike lists or tuples, sets do not allow duplicate values i.e. each element in a set must be unique. Sets are mutable, meaning you can add or remove items after a set has been created.

Sets are defined using curly braces `{}` or the built-in **set()** function. They are particularly useful for membership testing, removing duplicates from a sequence, and performing common mathematical set operations like union, intersection, and difference.

A set refers to a collection of distinct objects. It is used to group objects together and to study their properties and relationships. The objects in a set are called elements or members of the set.

Creating a Set in Python

Creating a set in Python refers to defining and initializing a collection of unique elements. This includes specifying the elements that will be part of the set, ensuring that each element is unique within the set.

You can create a set in Python using curly braces `{}` or the `set()` function –

Using Curly Braces

You can directly define a set by listing its elements within curly braces, separating each element by a comma as shown below –



```
my_set = {1, 2, 3, 4, 5}
print (my_set)
```

It will produce the following result –

```
{1, 2, 3, 4, 5}
```

Using the set() Function

Alternatively, you can create a set using the set() function by passing an iterable (like a list or a tuple) containing the elements you want to include in the set –



```
my_set = set([1, 2, 3, 4, 5])
print (my_set)
```

We get the output as shown below –

```
{1, 2, 3, 4, 5}
```

Duplicate Elements in Set

Sets in Python are unordered collections of unique elements. If you try to create a set with duplicate elements, duplicates will be automatically removed –



```
my_set = {1, 2, 2, 3, 3, 4, 5, 5}
print (my_set)
```

The result obtained is as shown below –

```
{1, 2, 3, 4, 5}
```

Sets can contain elements of different data types, including numbers, strings, and even other sets (as long as they are immutable) –

```
mixed_set = {1, 'hello', (1, 2, 3)}  
print (mixed_set)
```

The result produced is as follows –

```
{1, 'hello', (1, 2, 3)}
```

In Python, sets support various basic operations that is used to manipulate their elements. These operations include adding and removing elements, checking membership, and performing set-specific operations like union, intersection, difference, and symmetric difference.

Adding Elements in a Set

To add an element to a set, you can use the `add()` function. This is useful when you want to include new elements into an existing set. If the element is already present in the set, the set remains unchanged –

```
my_set = {1, 2, 3, 3}  
# Adding an element 4 to the set  
my_set.add(4)  
print (my_set)
```

Following is the output obtained –

```
{1, 2, 3, 4}
```

Removing Elements from a Set

You can remove an element from a set using the `remove()` function. This is useful when you want to eliminate specific elements from the set. If the element is not present, a `KeyError` is raised –

```
my_set = {1, 2, 3, 4}
# Removes the element 3 from the set
my_set.remove(3)
print (my_set)
```

The output displayed is as shown below –

```
{1, 2, 4}
```

Alternatively, you can use the **discard()** function to remove an element from the set if it is present. Unlike `remove()`, `discard()` does not raise an error if the element is not found in the set –



```
my_set = {1, 2, 3, 4}
# No error even if 5 is not in the set
my_set.discard(5)
print (my_set)
```

We get the output as shown below –

```
{1, 2, 3, 4}
```

Membership Testing in a Set

Sets provide an efficient way to check if an element is present in the set. You can use the **in** keyword to perform this check, which returns **True** if the element is present and **False** otherwise –



```
my_set = {1, 2, 3, 4}
if 2 in my_set:
    print("2 is present in the set")
else:
    print("2 is not present in the set")
```

Following is the output of the above code –

```
2 is present in the set
```

Set Operations

In Python, sets support various set operations, which is used to manipulate and compare sets. These operations include union, intersection, difference, symmetric difference, and subset testing. Sets are particularly useful when dealing with collections of unique elements and performing operations based on set theory.

- **Union** – It combine elements from both sets using the `union()` function or the `|` operator.
- **Intersection** – It is used to get common elements using the `intersection()` function or the `&` operator.
- **Difference** – It is used to get elements that are in one set but not the other using the `difference()` function or the `-` operator.
- **Symmetric Difference** – It is used to get elements that are in either of the sets but not in both using the `symmetric_difference()` method or the `^` operator.

Python Set Comprehensions

Set comprehensions in Python is a concise way to create sets based on iterable objects, similar to list comprehensions. It is used to generate sets by applying an expression to each item in an iterable.

Set comprehensions are useful when you need to create a set from the result of applying some operation or filtering elements from another iterable.

Syntax

The syntax for set comprehensions is similar to list comprehensions, but instead of square brackets `[ ]`, you use curly braces `{ }` to denote a set –

```
set_variable = {expression for item in iterable if condition}
```

Example

In the following example, we are creating a set containing the squares of numbers from 1 to 5 using a set comprehension –

```
squared_set = {x**2 for x in range(1, 6)}  
print(squared_set)
```

The output obtained is as follows –

```
{1, 4, 9, 16, 25}
```

Filtering Elements Using Set Comprehensions

You can include conditional statements in set comprehensions to filter elements based on certain criteria. For instance, to create a set of even numbers from 1 to 10, you can use a set comprehension with an if condition as shown below –

```
even_set = {x for x in range(1, 11) if x % 2 == 0}  
print(even_set)
```

This will produce the following output –

```
{2, 4, 6, 8, 10}
```

Nested Set Comprehensions

Set comprehensions also support nested loops, allowing you to create sets from nested iterables. This can be useful for generating combinations or permutations of elements.

Example

```
nested_set = {(x, y) for x in range(1, 3) for y in range(1, 3)}  
print(nested_set)
```

Output of the above code is as shown below –

```
{{(1, 1), (1, 2), (2, 1), (2, 2)}}
```

Frozen Sets

In Python, a frozen set is an immutable collection of unique elements, similar to a regular set but with the distinction that it cannot be modified after creation. Once created, the elements within a frozen set cannot be added, removed, or modified, making it a suitable choice when you need an immutable set.

You can create a frozen set in Python using the `frozenset()` function by passing an iterable (such as a list, tuple, or another set) containing the elements you want to include in the frozen set.

Example

In the following example, we are creating a frozen set of integers and then adding an element to it –

```
my_frozen_set = frozenset([1, 2, 3])
print(my_frozen_set)
my_frozen_set.add(4)
```

Following is the output of the above code –

```
frozenset({1, 2, 3})
Traceback (most recent call last):
  File "/home/cg/root/664b2732e125d/main.py", line 3, in <module>
    my_frozen_set.add(4)
AttributeError: 'frozenset' object has no attribute 'add'
```

TOP TUTORIALS

Python Tutorial

Java Tutorial

C++ Tutorial

C Programming Tutorial

C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics
Artificial Intelligence And Machine Learning
DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler

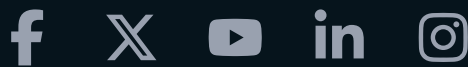


Chapters ▾

Categories ≡

Online SQL Compiler
Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.